Day 1 | Azure components, one at a time

Today's promise

- Understand the service before connecting it to an app.
- See the same resource in the portal and VS Code.
- Leave with repeatable, evidence-based exercises.

Our working rule

- Explain -> portal -> VS Code -> verify -> practise.
- Use existing, approved training resources.
- Fictional data only; no secrets on screen.

REFERENCE / Start Here: Phase A

Attendee route

CONCEPT

Two days: understand first, integrate later

Day 1 | fundamentals

- Orientation and Azure access
- SQL and Blob Storage
- HTTP Function, Logs and basic KQL

Day 2 | connections and operations

- Service Bus and real application telemetry
- Realistic document-tracker workflow
- CI/CD, cost and optional IaC / automation

Day 2 live integration is conditional on a prepared, tested application.

REFERENCE / Phases A, B and C

[Attendee route]

CONCEPT

Day 1 route | protect hands-on time

First half

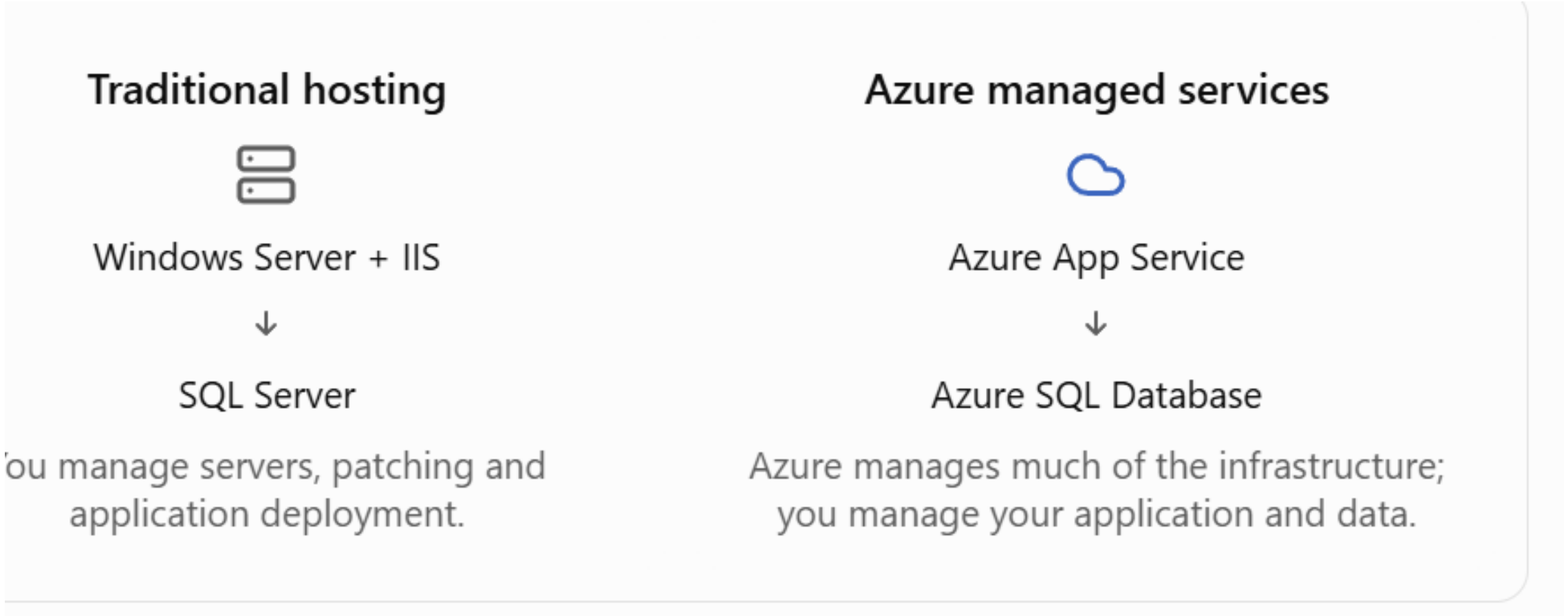
09:45-11:05 welcome + foundations + orientation
11:05-12:20 SQL + paired cost decision
12:20-12:35 break; 12:35-13:40 Blob
13:40-14:25 lunch

Second half

14:25-15:40 Function; 15:40-15:55 break
15:55-16:50 Logs / KQL
16:50-17:30 recap, costs and cleanup
Optional UI only if time permits

CONCEPT

You know .NET. Where would you host it?



Managed services reduce server administration; you still manage application, data, access and cost.

CONCEPT

Azure names | connect them to familiar ideas

Organisation and resources

- **Tenant:** your identity directory.
- **Subscription:** billing and management boundary.
- **Resource group (RG):** groups related resources.
- **Resource:** an actual database, app or service.

Location and access

- **Region:** geographic hosting area.
- **Zone:** separate datacentre group in a region.
- **Portal:** website for managing Azure.
- **RBAC:** who can do what, at which scope.

CONCEPT

One fictional mortgage application | which service?

Run the application and its work

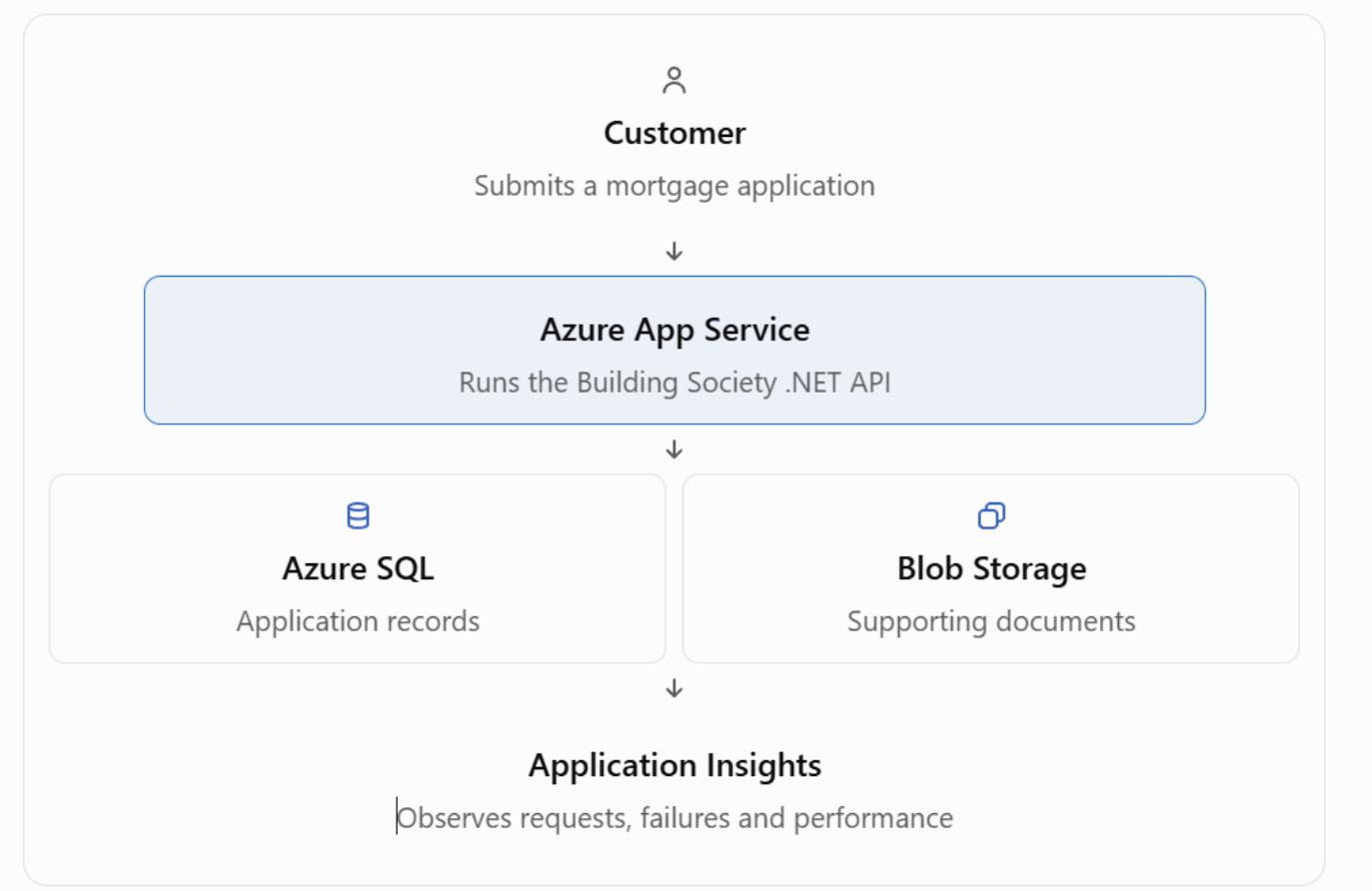
- .NET API -> **App Service.**
- Application records -> **Azure SQL.**
- PDF documents -> **Blob Storage.**
- Small event-driven code -> **Functions.**
- Queued background work -> **Service Bus.**

Protect, observe and control it

- Secrets, keys, certificates -> **Key Vault.**
- Password-free app identity -> **Managed Identity.**
- Requests and failures -> **Application Insights.**
- Analyse monitoring data -> **Logs / KQL.**
- Understand spending -> **Cost Management.**

CONCEPT

A customer's request | the services work together



Fictional example. Monitoring runs alongside the app, not as the next request step. Labs test components separately.

CONCEPT

Your turn | choose the service

Think in pairs | 5 minutes

1. Where would you run a **.NET API**?
2. Where would you store **application records**?
3. Where would you store an **uploaded PDF**?

Discuss together | 5 minutes

4. Access storage **without an app password**?
5. Investigate a **failed API request**?
6. Check **how much Azure is costing**?

Explain why, not just the service name.

CONCEPT

Before any write, check the target

Confirm every time

- Tenant, subscription and resource name
- Assigned table / container / Function host
- Your identity, network path and permission

Never use a shortcut

- No broad firewall rules to fix a login failure.
- No shared admin passwords or keys in evidence.
- No class-wide concurrent publish to one app.

REFERENCE / Instructor checklist and component checkpoints

Attendee route

CONCEPT

Azure names: where does each thing belong?

Management hierarchy

- Tenant: identity directory.
- Subscription: resource and billing boundary.
- Resource group: lifecycle grouping.
- Resource: server, account, app or workspace.

Developer perspective

- Code runs on your laptop OR a cloud host.
- A SQL server is not your local SQL Express.
- A workspace name is not its Workspace ID.
- Seeing Overview does not prove data access.

IaaS, PaaS and SaaS | the pizza comparison

Pizza analogy: how much do YOU still manage?

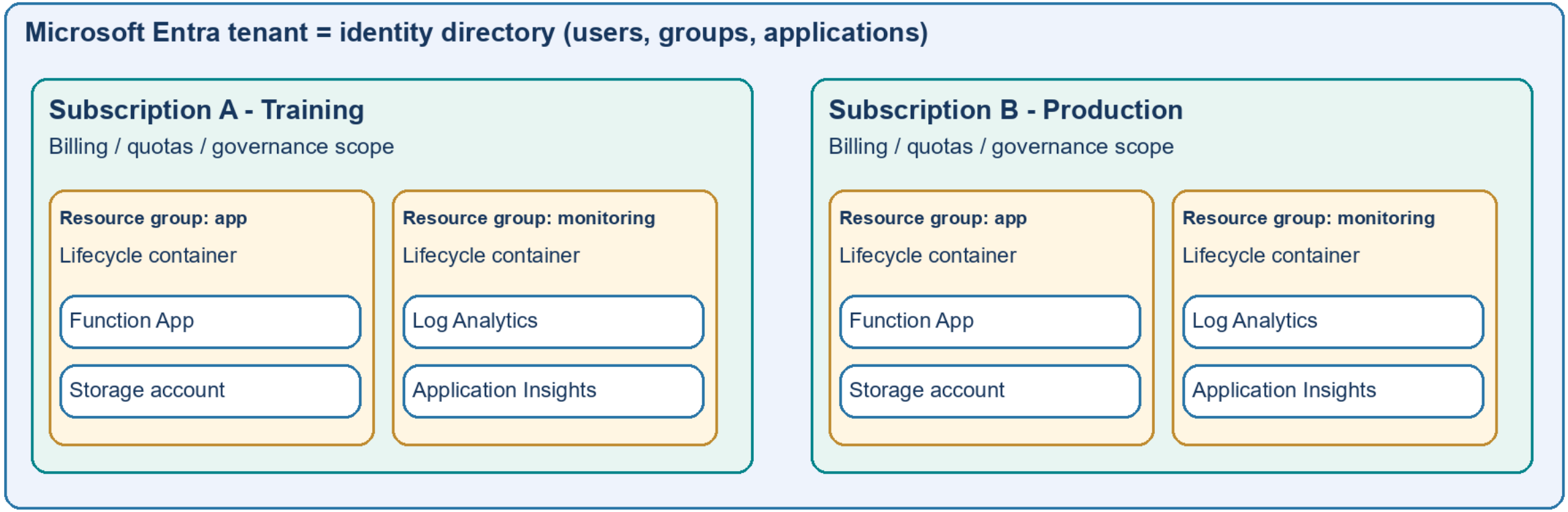
On-premises	IaaS	PaaS	SaaS
Make at home	**Rent a kitchen**	**Order delivery**	**Dine at a restaurant**
Own the kitchen, oven and ingredients.	Provider supplies kitchen; you cook and maintain what you install.	Provider cooks the pizza; you choose toppings and how to use it.	Use a finished service; you choose, configure and control your access.
Your own datacenter	**Azure Virtual Machines**	**App Service / Azure SQL**	**Microsoft 365**
Data, users and access	Data, users and access	Data, users and access	Data, users and access
Application code	Application code	Application code	Application code
Runtime / middleware	Runtime / middleware	Runtime / middleware	Runtime / middleware
Guest operating system	Guest operating system	Guest operating system	Guest operating system
Virtualization	Virtualization	Virtualization	Virtualization
Physical servers	Physical servers	Physical servers	Physical servers
Building / network	Building / network	Building / network	Building / network

Gold = customer-led work **Green = provider-led work** | **Simplified model: exact duties vary by service.**

Less infrastructure work does NOT mean no security work: protect identities, data, configuration and permissions.

Tenant, subscription, resource group and resource

Azure organisation: identity directory -> billing scope -> lifecycle group -> resource



A subscription trusts one tenant at a time; a tenant can be associated with many subscriptions.

Each resource belongs to one resource group. Groups may hold resources in different Azure regions.

Optional management groups sit above subscriptions. This picture is organisation, not a network or request flow.

Security | check the network, identity and permissions

Security: independent checks, not one magic permission

1. Network

Can the request reach it?

Firewall / private endpoint
Approved VPN / DNS route

2. Identity

Who is calling?

User sign-in / managed identity
Authenticate with approved method

3. Permission

May this identity do this?

Scoped Azure data role
SQL user + table permission

4. Protection

Is data handled safely?

TLS / private data / safe logs
Never share secrets or keyed URLs

Least privilege: scope the role to the smallest approved target

Subscription > resource group > resource > supported child scope (for example one Blob container)

Azure RBAC assignments normally inherit downward; deny assignments, conditions and service rules still apply.

Azure Reader != Blob data access != SQL table permission. A network allow-list grants no data permission.

Function key: a shared endpoint secret, NOT a person's sign-in. Regenerate exposed keys with the owner.

A 403 alone does not identify the failed check. Diagnose the exact error; do not broaden access blindly.

CONCEPT

Two gates: network reachability + permission

Network gate

- SQL: approved single-IP or private-network path.
- Blob: account firewall / private endpoint policy.
- A TCP connection alone is not a successful SELECT.

Identity / data gate

- Azure Reader does not grant SQL table access.
- Blob needs a scoped data role.
- Portal, CLI and MSSQL sign-in are separate.

REFERENCE / SQL 2A; Blob 1A-1B

SQL handout

Blob handout

DEMO

Demo 01 | one resource, two views

Portal

- Sign in; confirm directory and subscription.
- Open Training; identify SQL, Storage and Function.
- Record resource name and region; change nothing.

VS Code

- Open the supplied lab folder; check Get-Location.
- Azure Resources: sign in, select subscription, Refresh.
- A new window may keep your old chat in the original window.

REFERENCE / Function guide Parts 1-2; orientation-only prerequisites

Function handout

Attendee route

EXERCISE

Exercise 01 | prove your context

Attendee steps | 10 min

- Find your assigned subscription and resource group.
- Identify one resource in both tools.
- Open the correct lab folder; check Get-Location.

Checkpoint

- Record resource name and region.
- Record account / directory without secrets.
- Missing access? Mark Blocked; do not create a replacement.

REFERENCE / Start Here checkpoints; Function Part 2

Attendee route

Function handout

CONCEPT

Azure SQL: structure is not application code

What we change

- CREATE TABLE defines the structure.
- INSERT adds fictional rows.
- ALTER TABLE adds a column.
- UPDATE changes a targeted row.

What we do not deploy

- No new database is needed.
- No Web App / API integration today.
- Saving a .sql file does not execute it.
- MSSQL and portal are clients of one database.

REFERENCE / Main exercise and sections 6-7

SQL handout

Azure SQL | avoid paying for unnecessary capacity

What drives cost?

- Compute tier, allocated capacity and storage.
- Backup retention and optional replicas can add cost.
- Eligible serverless auto-pause stops compute billing while paused; storage still costs.

Attendee cost-saving habits

- Use the assigned database; do not create a duplicate.
- Use tiny tables and targeted queries.
- Do not change tiers or auto-pause on shared databases.
- Ask the owner about approved temporary-data cleanup.

Use approved existing resources. Ask before changing tiers, retention, capacity or shared settings.

Official service pricing / guidance (check current region and offer)

DEMO

Demo 02 | add an approved client IP

Portal | authorised operator

- Database -> Server link -> Security > Networking.
- With approval, choose Selected networks.
- Add current public IPv4; Start = End.
- Save, reopen and verify the exact rule.

Explain before testing

- Do not copy a screenshot's old IP.
- Private laptop IP is not public outbound IP.
- Leave Allow Azure services unchecked.
- Wait up to five minutes, then reconnect.

Instructor permission required; a firewall rule does not create a database user.

REFERENCE / Section 2A A-E: complete firewall walkthrough

SQL handout

DEMO

Demo 03 | create in portal, read in VS Code

Portal

- Connect to the assigned database, not master.
- Run the connection check; verify DB_NAME().
- Run 02-create-and-read.sql with your suffix.
- Expect exactly two fictional rows.

VS Code / MSSQL

- Create the approved server/database profile.
- Encrypt enabled; Trust server certificate=False.
- Open a SQL file; connect it; Execute Query.
- SELECT the same rows and compare.

REFERENCE / Sections 3-6; instructor SELECT is read-only

SQL handout

Read-only instructor SELECT

EXERCISE

Exercise 02 | modify and verify both ways

Do | 20 min

- Use only your assigned table suffix.
- Run 03-modify-and-check.sql in MSSQL.
- Keep GO between ALTER and later statements.
- Rerun the SELECT in portal Query editor.

Expected evidence

- Four columns, including nullable Notes.
- ID 1: Completed / Updated from VS Code.
- ID 2: Needs review / NULL.
- Optional: change ID 2 Notes in portal; read in MSSQL.

No WHERE, no targeted update: pause and inspect before execution.

REFERENCE / Section 7 and completion checklist

SQL handout

CONCEPT

SQL: classify the error before changing anything

Connectivity / startup

- Wrong hostname: copy from server Overview.
- Timeout: verify approved route and current IP.
- Paused serverless database: allow resume and retry.
- Blank portal blade: a portal-loading issue, not proof of SQL denial.

Authentication / execution

- Login failed: identity, tenant and database.
- Permission denied: scoped SQL rights.
- Wrong result: database, suffix and selected SQL text.
- Invalid Notes column: retain the GO batch boundary.

REFERENCE / Section 9 troubleshooting

SQL handout

CONCEPT

Blob Storage: private files, not a public website

Resource model

- Account -> container -> blob.
- A private container blocks anonymous reads.
- Network rules and Entra data roles are separate.

This exercise

- Tiny fictional .txt files only.
- Storage Blob Data Contributor at assigned container.
- CLI always uses --auth-mode login.
- No keys, SAS or Function bindings.

REFERENCE / Sections 1, 1A and 1B

Blob handout

Blob Storage | tiny files, appropriate access tiers

What drives cost?

- Stored bytes, operations and redundancy.
- Retrieval and data transfer can add charges.
- Cool, cold and archive may have minimum retention / early deletion charges.

Attendee cost-saving habits

- Use tiny fictional files in your assigned container.
- Hot is appropriate for this short, frequently accessed lab.
- Avoid repeated downloads and unnecessary copies.
- Delete only your lab data after owner approval.

Use approved existing resources. Ask before changing tiers, retention, capacity or shared settings.

Official service pricing / guidance (check current region and offer)

DEMO

Demo 04 | prepare access without widening it

Portal checks

- Account > Networking: all / selected / disabled?
- Selected networks: approved exact public IP only.
- IAM: verify container-scoped data role.
- Blob view: switch to Microsoft Entra user account.

Explain the boundaries

- Private container != private endpoint.
- If all networks allowed, an extra IP is not needed.
- Never tighten shared account networking mid-demo.
- Wait for role propagation; do not add broader roles.

REFERENCE / Sections 1A-1B; network and IAM walkthroughs

Blob handout

DEMO

Demo 05 | portal upload -> terminal download

Portal

- Open your assigned private container.
- Upload Samples\Storage\portal-note.txt.
- Leave overwrite off; Refresh after success.
- Confirm the file and nonzero byte length.

VS Code terminal

- Sign Azure CLI in separately; confirm subscription.
- Set $account and $container to assigned values.
- List and download using --auth-mode login.
- Compare source and downloaded SHA256 hashes.

REFERENCE / Sections 2-6; copy exact commands from the handout

Blob handout

EXERCISE

Exercise 03 | terminal upload -> portal verify

Do | 20 min

- Open and save vscode-note.txt.
- Upload to your own container; no blind overwrite.
- Check CLI exit code and list both names.
- Refresh portal; inspect/download the new file.

Checkpoint

- Refresh the assigned container; confirm the uploaded name.
- Download; copy its real path from browser Downloads.
- Get-FileHash: compare full SHA256 with the exact source.
- Repeat for both directions; no blind overwrite or deletion.

REFERENCE / Sections 7-8; repeat-run and cleanup guidance

Blob handout

CONCEPT

HTTP Function: code that has a cloud host

Local concepts

- HelloAzureFunction: project folder.
- ApplicationStatus: the function name and route.
- Core Tools runs the local Functions host.
- Azurite is local development storage.

Cloud concepts

- Function App: approved deployment target.
- .NET 10 isolated runtime must match the project.
- Function-level key protects this training endpoint.
- This fixed JSON does not read SQL or Blob.

REFERENCE / Names, prerequisites and Parts 3-5

Function handout

Azure Functions | local testing is not Azure shutdown

What drives cost?

- Hosting plan, executions and execution resources.
- Always-ready instances can add baseline charges.
- Host storage, logs and network services have separate costs.

Attendee cost-saving habits

- Build and test locally first; avoid repeated cloud calls.
- Reuse the approved host; no duplicate plan.
- Ctrl+C stops only the local server, not Azure charges.
- Do not change shared scaling / always-ready settings.

Use approved existing resources. Ask before changing tiers, retention, capacity or shared settings.

Official service pricing / guidance (check current region and offer)

DEMO

Demo 06 | build, host, then request

Terminal 1

- Open HelloAzureFunction by itself.
- Create local settings from supplied example if absent.
- Build; start Azurite.
- func start --port 7071; KEEP IT RUNNING.

Terminal 2

- Request /api/ApplicationStatus on localhost:7071.
- Check reference, Received, fictionalData and message.
- After testing: Ctrl+C in terminal 1; wait for prompt.
- Local host stops; Azure host and source code remain.

Build alone is not an HTTP test. Verify your own local response; Ctrl+C stops only your local host.

REFERENCE / Part 3: local settings, host and second-terminal test

Function handout

DEMO

Demo 07 | publish only to an approved host

VS Code publishing route

- Confirm target name, runtime and replacement approval.
- Use the supplied Function project and publish task.
- Choose the assigned Function App, not a Web App.
- Wait for success, then verify the deployed function.

Cloud verification

- Get Function URL: choose default (Function key).
- Not Host key or _master; paste into browser, not chat.
- Keyed: expected JSON; Test/Run can show HTTP 200.
- Remove ?code=...: expect HTTP 401, not a broken app.

REFERENCE / Parts 4-5; test-only route if publish not approved

Function handout

EXERCISE

Exercise 04 | record local and cloud separately

Do | 20 min

- Run the supplied local Function when tools are ready.
- Publish only with target-owner approval.
- Otherwise test the approved existing cloud endpoint.
- Record status code and response fields, never keys.

Checkpoint

- Local host: Passed / Blocked / Not run.
- Publish: Passed / Not approved / Not run.
- Cloud: expected response and keyless denial.
- Website results are Not run in Function-only route.

REFERENCE / Parts 3-5 and Function-related Part 8 results

Function handout

CONCEPT

Log Analytics is the service; KQL is the language

Workspace

- Holds queryable log tables.
- Has a name, resource ID and Workspace ID GUID.
- Query access is separate from Overview access.

KQL

- Pipe operators transform tabular results.
- where filters; project selects columns.
- order by sorts; summarize aggregates.
- A saved .kql file is not an ingested log.

REFERENCE / Sections 1-3

Logs / KQL handout

Log Analytics | collect what you actually need

What drives cost?

- Log ingestion and retention beyond included allowances.
- Some table plans charge for queries / search.
- Export, alerts and other monitoring features may add cost.

Attendee cost-saving habits

- Use the supplied synthetic query; it ingests no rows.
- For real logs, use a short time range and relevant scope.
- Do not enable verbose logging or new collection just for practice.
- Ask the owner before changing retention or caps.

Use approved existing resources. Ask before changing tiers, retention, capacity or shared settings.

Official service pricing / guidance (check current region and offer)

DEMO

Demo 08 | start with a predictable query

Portal Logs

- Open assigned workspace; confirm query scope.
- Close Welcome and Queries hub (top-right X).
- Turn Agent off; select KQL mode above the editor.
- Paste supplied query; Run; expect three ordered rows.

Explain the result

- LBS-DEMO-1001: Completed.
- LBS-DEMO-1002: Needs review.
- LBS-DEMO-1003: Received.
- Rows are synthetic; no ingestion is proven.

REFERENCE / Sections 2-3; no ADX cluster required

Logs / KQL handout

Supplied KQL

CONCEPT

Read the pipeline from top to bottom

Supplied main query

```
datatable(Reference:string, Status:string)
[
  "LBS-DEMO-1001", "Completed",
  "LBS-DEMO-1002", "Needs review",
  "LBS-DEMO-1003", "Received"
]
| order by Reference asc
```

Try only in a separate portal tab

- Add a where filter for Completed: one row.
- Restore the original before terminal comparison.
- Keep supplied KQL and request JSON in sync.
- The main script asserts exactly three rows.

REFERENCE / Section 3; optional filter practice

Logs / KQL handout

Supplied KQL

EXERCISE

Exercise 05 | same query from VS Code

Do | 15 min

- Workspace Overview > Essentials > copy Workspace ID.
- Not workspace name, Subscription ID or resource ID.
- In PortalAndVsCodeBasics, use the signed-in CLI.
- Run the supplied script with that GUID; compare results.

Checkpoint

- Script prints three expected rows and PASS.
- Compare with portal Logs results.
- Record workspace and result time.
- No workspace key, agent or new table needed.

REFERENCE / Sections 4-5; exact script invocation

Logs / KQL handout

EXERCISE

Optional | dummy tracker UI is not integration

Local-only UI practice

- Open DummyDocumentTracker.
- Run on localhost:5203 using the guide.
- Check three records and detail buttons.

Say this explicitly

- Fixed sample records, not SQL rows.
- No upload/processing or Blob integration.
- The real local scenario is a different project.
- No website deployment required today.

REFERENCE / Parts 6-8 only; optional local UI

Function handout

App Service / local websites | understand the hosting boundary

What drives cost?

- Paid App Service plans charge for allocated instances.
- Stopping an app does not stop paid plan charges.
- Related storage, monitoring and network services can still cost.

Attendee cost-saving habits

- Run the optional dummy UI locally; Azure hosting is not required.
- Never create a paid Web App without approval.
- The Function FC1 plan is not the website's hosting target.
- Only the owner approves plan deletion or resizing.

Use approved existing resources. Ask before changing tiers, retention, capacity or shared settings.

Official service pricing / guidance (check current region and offer)

CONCEPT

A repeatable troubleshooting ladder

Check in order

- 1. Correct folder, file and tool?
- 2. Correct tenant, subscription and target?
- 3. Reachable network and current IP?
- 4. Correct identity and scoped permission?

Then verify

- 5. Exact error / CLI exit code?
- 6. Small read-only request first.
- 7. Make one approved change; retest.
- 8. Save evidence without sensitive data.

REFERENCE / Check each service's troubleshooting section

Attendee route

EXERCISE

Day 1 exit ticket

Explain

- Why are network and identity separate gates?
- What executes when you save a SQL/KQL file?
- How is a private container different from a private endpoint?

Demonstrate

- Show one verified component in both tools.
- Record a blocked step accurately.
- Identify the correct guide and safe cleanup.
- Tomorrow: messaging, real telemetry and the whole workflow.

REFERENCE / Component checkpoints; retain your results

Attendee route

CONCEPT

Your Day 1 documents

Use the handouts for exact steps

- Start Here: order and checkpoints.
- SQL: firewall, MSSQL and modification scripts.
- Blob: network/IAM and two-way file checks.

Continue independently

- Function: local / approved publish / cloud checks.
- Logs/KQL: workspace and matching synthetic query.
- Illustrated Day 1 quick reference: diagrams + checklist.
- Record your own Passed / Blocked / Not run results.

REFERENCE / Clickable links below; keep deck beside documents

Attendee route | SQL handout | Blob handout | Function handout | Logs / KQL handout